

COMPRENDRE L'ARCHITECTURE DES WEB SERVICES REST



Objectifs du cours

□ REST

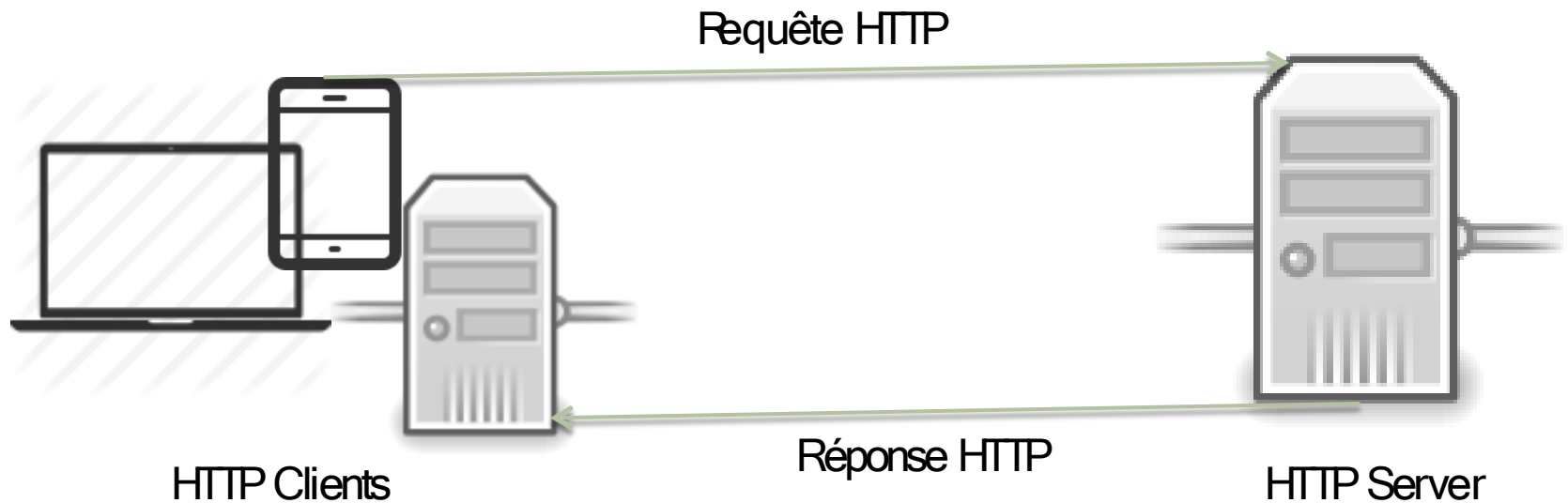
- ▮ Comprendre l'architecture des applications compatibles REST
- ▮ Exposer des services REST
- ▮ Consommer des services REST dans des applications Web



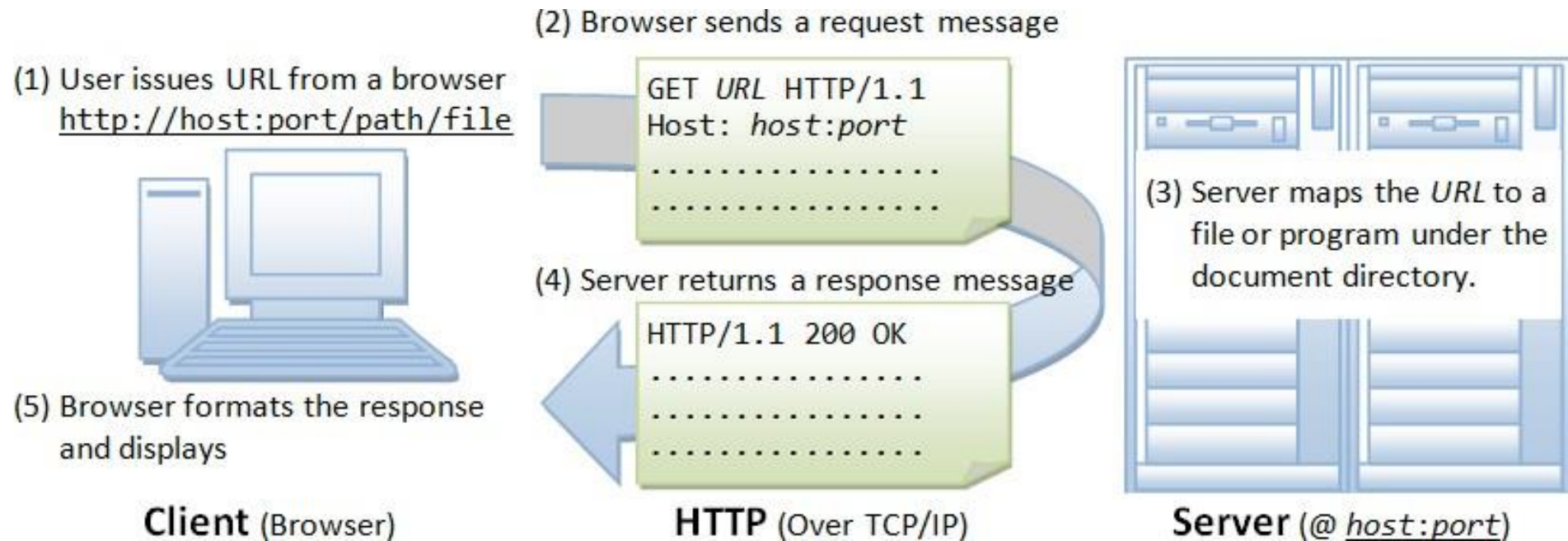
Rappel → Protocol HTTP

Le Protocole HTTP

- HyperText Transfer Protocol
- Protocole d'échanges d'information sur le web
- Basé sur TCP/IP



Enchainement Client Serveur



URL

- Unique Resource Location
- Identifie les ressources de manière unique sur le Web
- 4 parties
 - ▮ Protocole (http, ftp, mail, ...)
 - ▮ Host (google.com)
 - ▮ Port (8080, 80)
 - ▮ Path (Chemin vers la ressource sur le serveur)

Requêtes HTTP

- Permet à un client d'envoyer des messages à un serveur
- Format d'un message HTTP
 - ▮ Request Message Header
 - Request Line
 - Request Headers [Optional]
 - ▮ Request Message Body

Request Message Header

□ Request Line

```
POST / bibliotheque/faces/views/ categorie/ Create.xhtml HTTP/1.1
```

□ Request Headers

```
Host: localhost:8080
Connection: keep-alive
Content-Length: 176
Cache-Control: max-age=0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Origin: http://localhost:8080
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/39.0.2171.65 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Referer: http://localhost:8080/bibliotheque/faces/views/categorie/List.xhtml
Accept-Encoding: gzip, deflate
Accept-Language: fr,fr-FR;q=0.8,en;q=0.6
Cookie: JSESSIONID=d64a9484e61761662575b5d14af1
```

Request Message Body

- Contient les données supplémentaires envoyées au serveur

```
j_idt13:nom:Miage  
j_idt13:description:NTDP
```

Réponse HTTP

- Réponse du serveur au client
- Format d'une réponse HTTP
 - ▮ Response Message Header
 - Response Line
 - Response Headers
 - ▮ Response Message [Optional]

Response Message Header

□ Response Line

```
HTTP/1.1 200 OK
```

□ Response Headers

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.1 JSP/2.3 (GlassFish Server Open Source Edition 4.0 Java/Oracle Corporation/1.8)
Server: GlassFish Server Open Source Edition 4.0
Content-Type: text/html; charset=UTF-8
Date: Sun, 23 Nov 2014 16:05:39 GMT
Content-Length: 2274
```

Response Message Body

□ Response Body

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"><html xmlns="http://www.w3.org/1999/xhtml"><head><link
type="text/css" rel="stylesheet" href="/bibliotheque/faces/javafx.faces.resource/theme.css?ln=primefaces-aristo" /
><link type="text/css" rel="stylesheet" href="/bibliotheque/faces/javafx.faces.resource/css/jsfcrud.css" / >
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" / >
  <title>Create New Categorie</title></head><body>
  <h1>Create New Categorie
  </h1>
  <p><div id="messagePanel"><table><tr style="color: green;"><td>Categorie was successfully created. </td>
</tr></table></div>
</html>
</html>
```

Méthodes HTTP

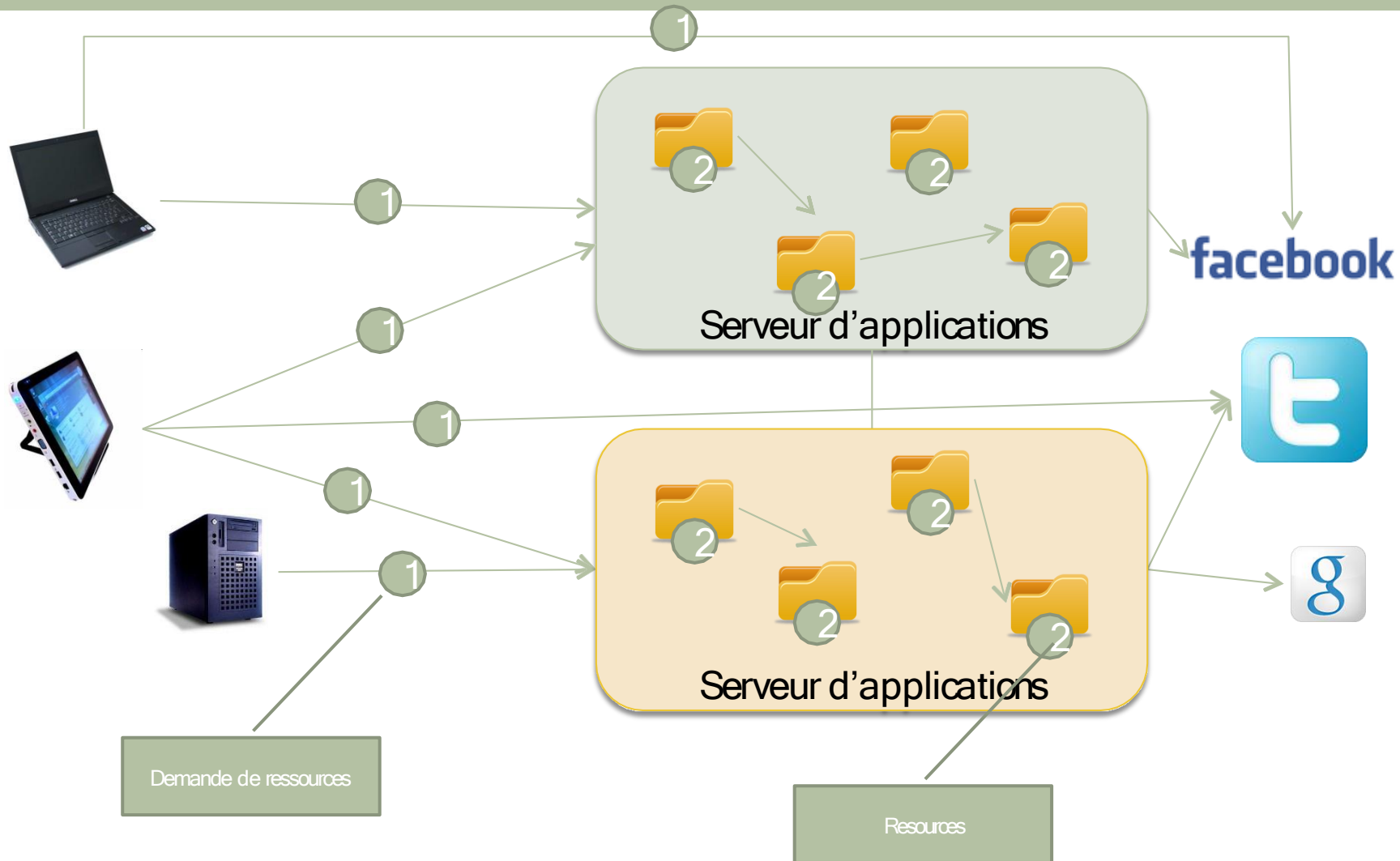
- HTTP définit un ensemble de méthode permet de caractériser les requêtes
 - ▮ GET : Récupérer des ressources à un serveur
 - ▮ POST : Envoyer des données à un serveur
 - ▮ PUT : Modifier des données
 - ▮ DELETE : Suppression de données
 - ▮ OPTIONS : Demander la liste des méthodes supportées par un serveur
 - ▮ Autres : HEAD, TRACE, CONNECT



Partie 1

Introduction aux Web Services

Utilisation du web aujourd'hui (1)



Web Services (Définition)

- ❑ Services informatiques de la famille des technologies web permettant la communication entre des applications **hétérogènes** dans des environnements distribués (*Wikipédia*).
- ❑ Ils ont été proposés à la base comme solution d'intégrations de différents logiciels développés par des entreprises (ERP, SCM, CRM) leur permettant de communiquer entre eux.
- ❑ Basés sur XML (description et échange) et utilisant en général les protocoles du web comme canal de communication;

Types de Services Web

- Deux principaux types
 - ▮ SOAP
 - ▮ REST



Services Web RESTFu

Web Service REST

Définition

- ❑ Acronyme de **RE**presentational **State** Transfert défini dans la thèse de Roy Fielding en 2000.
- ❑ **REST** n'est pas un protocole ou un format, contrairement à **SOAP**, **HTTP** ou **RCP**, mais un style d'architecture inspiré de l'architecture du web fortement basé sur le protocole **HTTP**
- ❑ Il n'est pas dépendant uniquement du web et peut utiliser d'autre protocoles que **HTTP**

Web Service REST

Ce qu'il est :

- Un système d'architecture
- Une approche pour construire une application

Ce qu'il n'est pas

- Un protocole
- Un format
- Un standard

REST → utilisation

- Utilisé dans le développement des applications orientés ressources (ROA) ou orientées données (DOA)
- Les applications respectant l'architecture REST sont dites RESTful

REST → Fournisseurs

facebook



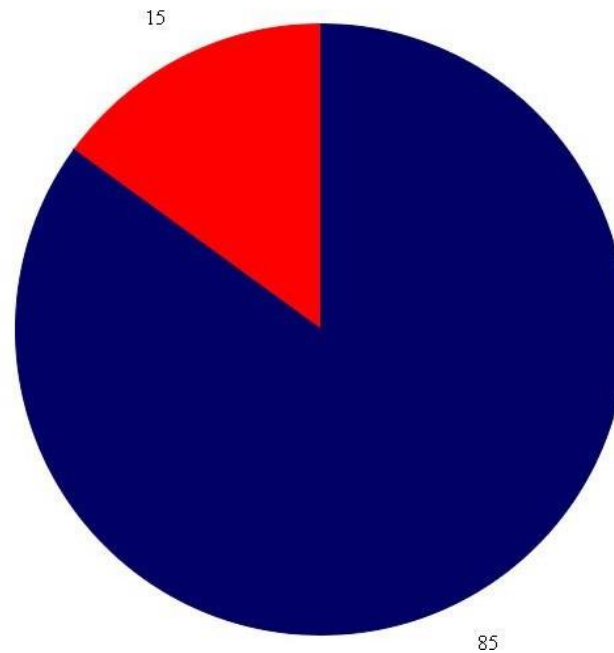
Google



REST → Statistics

Statistique d'utilisation des services web REST et SOAP chez AMAZON

■ REST ■ SOAP



REST → Caractéristiques

- Les services REST sont sans états (Stateless)
 - ▮ Chaque requête envoyée au serveur doit contenir toutes les informations relatives à son état et est traitée indépendamment de toutes autres requêtes
 - ▮ Minimisation des ressources systèmes (pas de gestion de session, ni d'état)
- Interface uniforme basée sur les méthodes HTTP (GET, POST, PUT, DELETE)
- Les architectures RESTful sont construites à partir de ressources uniquement identifiées par des URI(s)

Requêtes REST

- Ressources
 - ▮ Identifiée par une URI (<http://unice.fr/cursus/master/miage>)
- Méthodes (verbes) permettant de manipuler les ressources (identifiants)
 - ▮ Méthodes HTTP : GET, POST, PUT, DELETE
- Représentation : Vue sur l'état de la ressource
 - ▮ Format d'échanges entre le client et le serveur (XML, JSON, text/plain,...)

Ressources

- ❑ Une ressource est un objet identifiable sur le système

→ Livre, Catégorie, Client, Prêt

Une ressource n'est pas forcément un objet matérialisé (Prêt, Consultation, Facture...)

- ❑ Une ressource est identifiée par une URI : Une URI identifie uniquement une ressource sur le système

<http://ntdp.miage.fr/bookstore/books/1>

✓
Clef primaire de la
ressource dans la BDD

Méthodes (Verbes)

- Une ressource peut subir quatre opérations de bases CRUD correspondant aux quatre principaux types de requêtes HTTP (GET, PUT, POST, DELETE)
- RESTs'appuie sur le protocole HTTP pour effectuer ces opérations sur les objets
 - ▮ CREATE → POST
 - ▮ RETRIEVE → GET
 - ▮ UPDATE → PUT
 - ▮ DELETE → DELETE

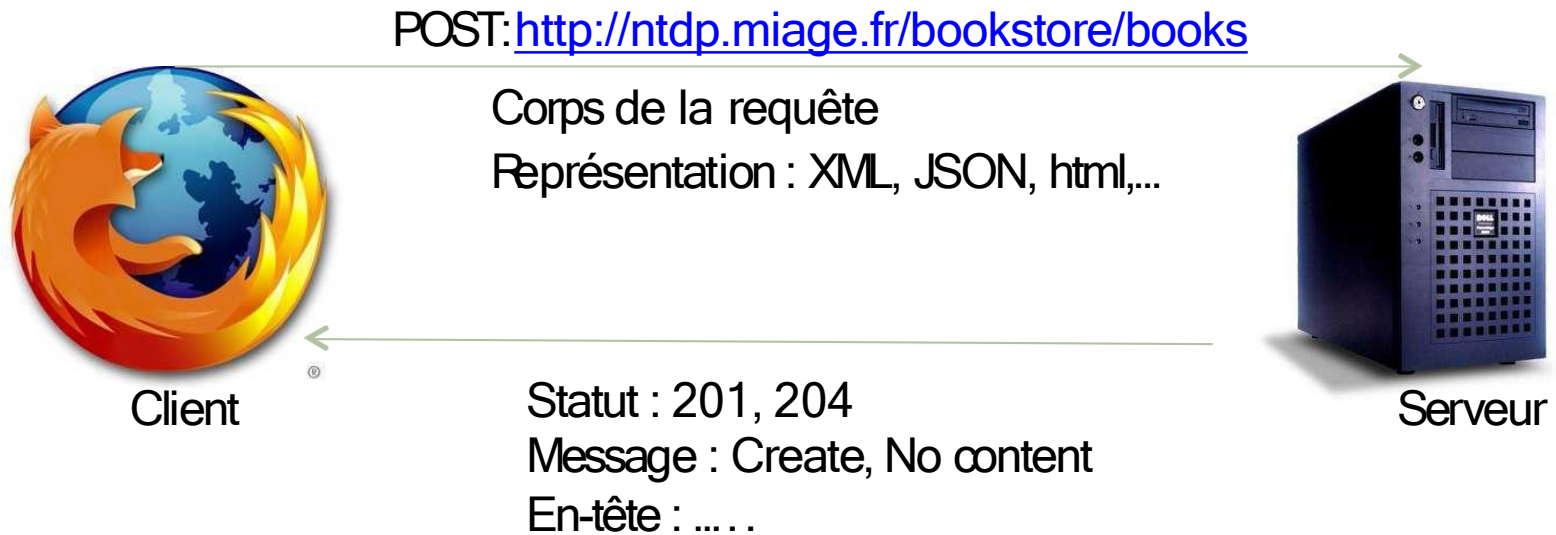
Méthode GET

- La méthode GET renvoie une représentation de la ressource tel qu'elle est sur le système



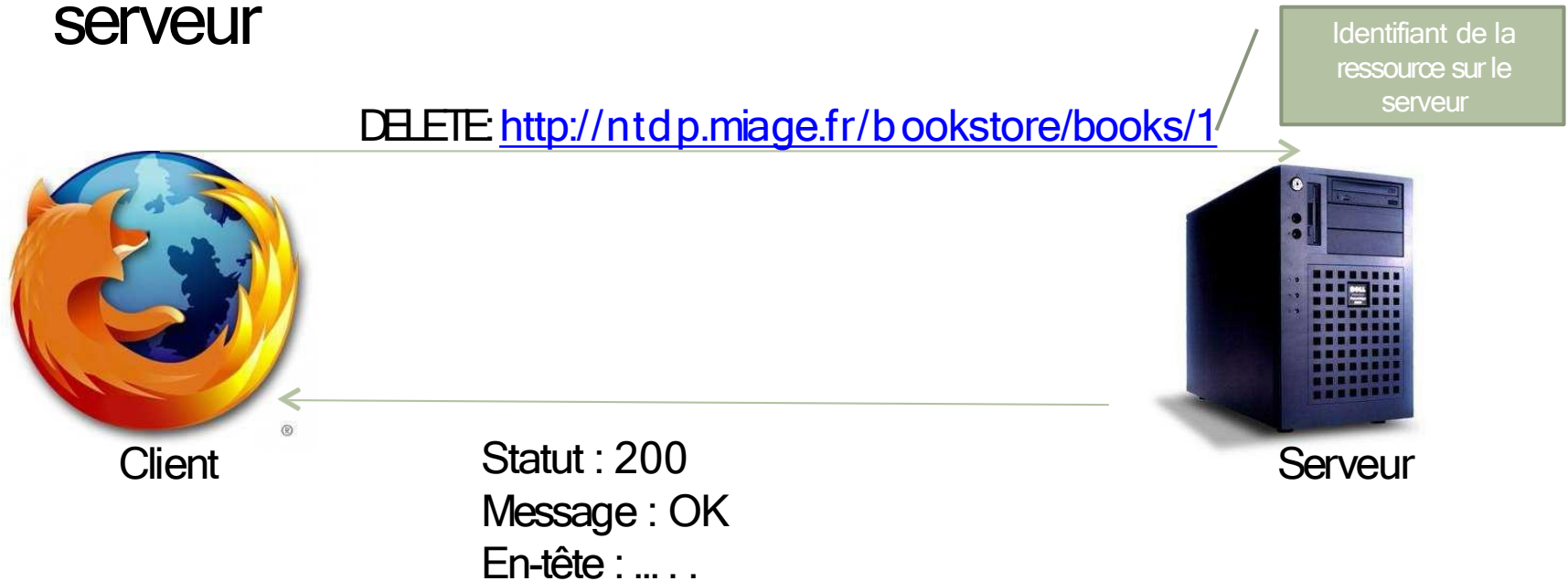
Méthode POST

- La méthode POST crée une nouvelle ressource sur le système



Méthode DELETE

- Supprime la ressource identifiée par l'URI sur le serveur



Méthode PUT

- Mise à jour de la ressource sur le système



Reflexions

- Que se passe t il
 - ▮ si on fait de la lecture avec un POST ?
 - ▮ Si on fait une mise à jour avec un DELETE ?
 - ▮ Si on fait une suppression avec un PUT ?
- REST ne l'interdit pas
- Mais si vous le faites, votre application ne respecte pas les exigences REST et donc n'est pas RESTful

Représentation

Une représentation désigne les données échangées entre le client et le serveur pour une ressource:

- ❑ HTTP GET → Le serveur renvoie au client l'état de la ressource
- ❑ PUT, POST → Le client envoie l'état d'une ressource au serveur

Peut être sous différent format :

- ❑ JSON
- ❑ XML
- ❑ XHTML
- ❑ CSV
- ❑ Text/plain
- ❑



Rappel → JSON

JSON

JSON « **J**ava**S**cript **O**bject **N**otation » est un format d'échange de données, facile à lire par un humain et interpréter par une machine.

Basé sur JavaScript, il est complètement indépendant des langages de programmation mais utilise des conventions qui sont communes à tous les langages de programmation (C, C++, Perl, Python, Java, C#, VB, JavaScript,...)

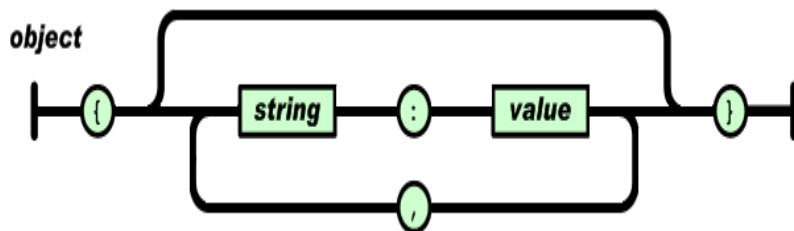
Deux structures :

- Une collection de defs/valeurs → Object
- Une collection ordonnée d'objets → Array

JSON

Objet

Commence par un « { » et se termine par « } » et composé d'une liste non ordonnée de paire def/ valeurs. Une def est suivie de « : » et les paires def/ valeur sont séparés par « , »

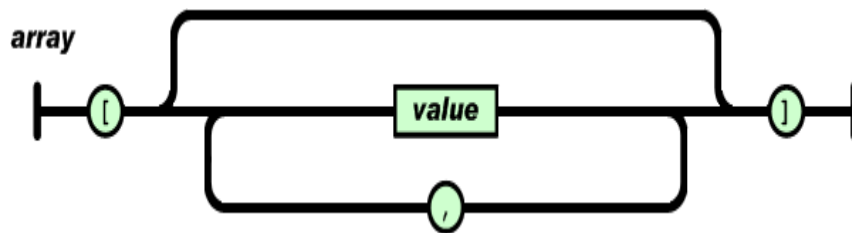


```
{ "id": 51,
  "nom": "Mathematiques 1", "resume":
  "Resume of math ", "isbn": "123654",
  "categorie":
    {
      "id": 2, "nom": "Mathematiques",
      "description": "Description of
      mathematiques "
    },
  "quantite": 42,
  "photo": ""
}
```

JSON

ARRAY

Liste ordonnée d'objets commençant par « [» et se terminant par «] », les objets sont séparés l'un de l'autre par « , ».

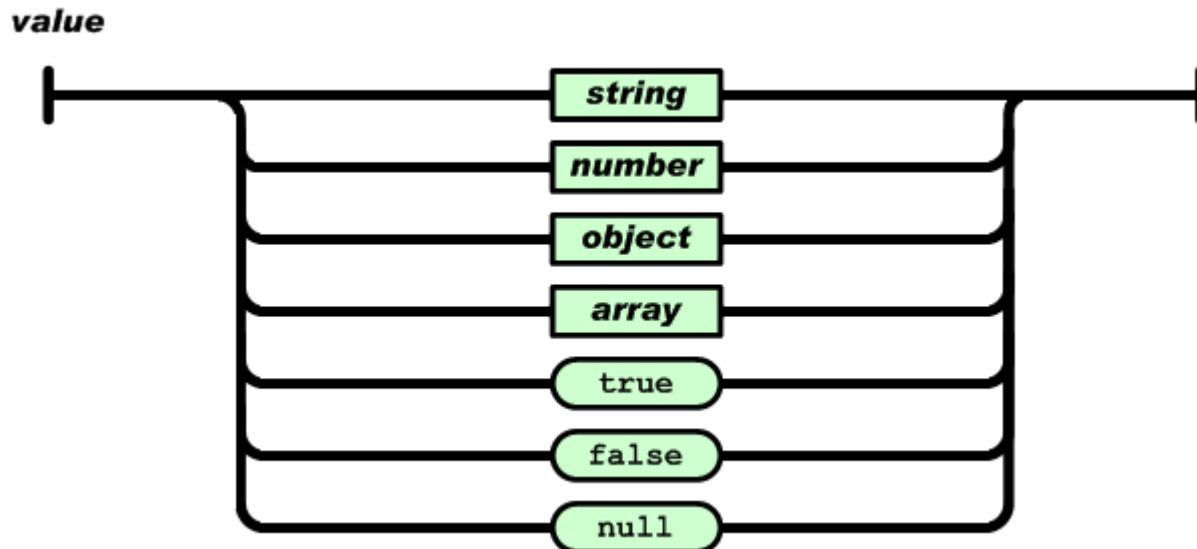


```
[  
  { "id": 51,  
    "nom": "Mathematiques 1",  
    "resume": "Resume of math ",  
    "isbn": "123654",  
    "quantite": 42,  
    "photo": ""  
  },  
  { "id": 102,  
    "nom": "Mathematiques 1",  
    "resume": "Resume of math ",  
    "isbn": "12365444455",  
    "quantite": 42,  
    "photo": ""  
  }  
]
```

JSON

Value

Un objet peut être soit un string entre « "" » ou un nombre (entier, décimal) ou un boolean (true, false) ou null ou un objet.



Services Web étendus VS REST

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body> <ns2:hello xmlns:ns2="http://
services.bibliotheque.ntdp.miage.unice.fr/">
    <name>Miage NTDP</name>
  </ns2:hello>
</S:Body>
</S:Envelope>
```



Client

SOAP



Serveur



Client

<http://localhost:8080/Bibliotheque/webresources/category/Miage%20NTDP>

REST



Serveur

Services Web étendus VS REST

SOAP

→ Avantages

- Standardisé
- Interopérabilité
- Sécurité (WS-Security)

→ Inconvénients

- Performances (enveloppe SOAP supplémentaire)
- Complexité, lourdeur
- Cible l'appel de service

Services Web étendus VS REST

REST

→ Avantages

- Simplicité de mise en œuvre
- Lisibilité par un humain
- Evolutivité
- Repose sur les principes du web
- Représentations multiples (XML, JSON,...)

→ Inconvénients

- Sécurité restreinte par l'emploi des méthodes HTTP
- Cible l'appel de ressources



A vos marques !